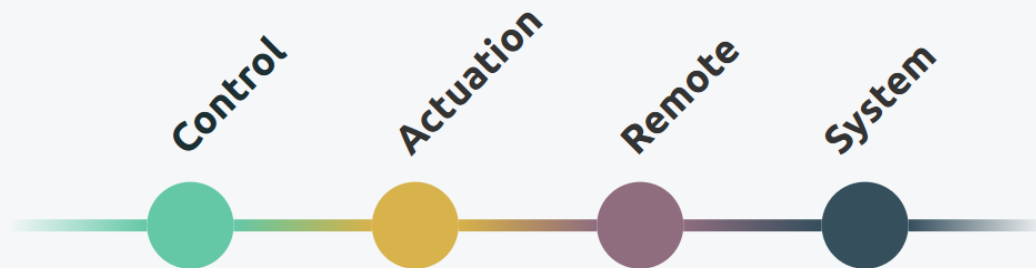


PROJECT C.A.R.S.

Control and Actuation via Remote Systems

A multi-node wireless rover control system utilizing ESP-NOW and UART synchronization.



AUTHOR

Lawton Jones
Mechanical Engineering

COURSE

CMPE 3815: Microcontroller Systems
University of Vermont

Contents

1	Executive Summary	2
2	Introduction	2
2.1	Project Vision	2
2.2	Problem Statement	2
3	Design Requirements	2
4	Hardware Architecture	2
4.1	Component Overview	2
4.2	System Architecture	3
4.3	System Overview	3
4.4	The Functional Stages	3
4.4.1	1: Remote Sensing and Data Packaging:	3
4.4.2	2: Wireless Communication Bridging:	3
4.4.3	3: Motor Control and Actuation:	3
4.5	Wiring Schematics	4
5	Stage 1: Remote Sensing and Data Packaging	6
5.1	The Pedal Module	7
5.2	The Steering Module	8
5.3	Sensor Fusion	9
5.4	Asynchronous Transmission	9
6	Stage 2: Communication Bridge	10
6.1	Communication Bridge and Signal Integration	11
6.2	Wireless Protocol: ESP-NOW	11
6.3	Data Structure and Encapsulation	11
6.4	Asynchronous Receiving and Callback Logic	11
6.5	Serial Packaging	12
6.6	Logic Level Shifting and Voltage Divider	12
7	Stage 3: Motor Control and Actuation	12
7.1	Steering Control Logic and Input Mapping	13
7.2	Arduino Integration via Software Serial	14
7.2.1	Interface Configuration	14
7.2.2	Parsing Serial Data	14
7.2.3	Signal Integrity and Logic Shifting	14
8	Project Management	15
8.1	Timeline	15
8.2	Initial Planned Timeline	15
9	Conclusion	16
9.1	Results	16
9.2	Final Reflection	16
	Appendix	18
A	Microcontroller Source Codes	18
A1	Pedal Controller Transmitter	18
A2	Steering Wheel Transmitter	21
A3	Rover Receiver and Serial Bridge	26
A4	Rover Motor Actuation and Driving Logic	29
B	Initial Project Proposal	34

EXECUTIVE SUMMARY

The primary objective of Project CARS was to design and implement a wireless control system for the Lafvin Mecanum Car using ESP32 microcontrollers and their wireless communication protocol ESP-NOW. With a goal of implementing realistic driving controls utilizing pedals and an IMU based steering wheel as the main interfacing components. The goals of the project are clearly defined in the Design Requirements section. Over the course of a month, 4 of the 5 requirements were met, only missing the Fail-Safe requirement. A full list of the completed components can be found in the Timeline section.

INTRODUCTION

PROJECT VISION

The vision for Project CARS was to create a rover that could be controlled and actuated through multiple wireless input sources, utilizing simulation pedals and an IMU based steering wheel. The goal was to learn how to interface with unknown devices and understand wireless communication through ESP32 microcontrollers. All while handling multi-node data for real time actuation.

PROBLEM STATEMENT

Standard remote-controlled systems often suffer from simplistic input mapping and unrealistic input physics. The goal of Project CARS is to create a wireless controlled rover that emulates real pedals and steering wheel physics.

DESIGN REQUIREMENTS

Project CARS functional requirements and technical constraints

- **Wireless Communication:** Implement wireless communication from control input to rover output through the ESP32.
- **Speed Mapping:** Map input from Pedals to rover speed control. Speed is variable: small pedal angle → slow speed, large pedal angle → fast speed.
- **Proportional Steering Mapping:** Proportionally map steering wheel angle input to rover turn radius. Steering feels smooth and not choppy.
- **Fail-Safe:** Implement fail safe to stop rover if wireless connection is lost.
- **Sensor Drift Accuracy and Steering Calibration:** Implement steering zeroing reset button and correctly handle drift from MPU by maintaining $\pm 2^\circ$ over 5 min of driving.

HARDWARE ARCHITECTURE

Project CARS is a project that utilizes three modules and four microcontrollers with each microcontroller having a unique program in order for the system to work cohesively. The full programs can be seen in the Appendix.

COMPONENT OVERVIEW

- **Microcontroller:** 3x ESP32's, 1x Arduino UNO
- **Input Devices:** Logitech PS2 Sim Pedals, MPU 6500
- **Output Devices:** 4 DC Motors
- **Power:** 2x Power Supply Modules
- **Passive Components:** Motor control shield and 4WD Mecanum Robot Car Kit(Lafvin)
- **Prototyping:** Breadboard and Jumper Wire Kit

SYSTEM ARCHITECTURE

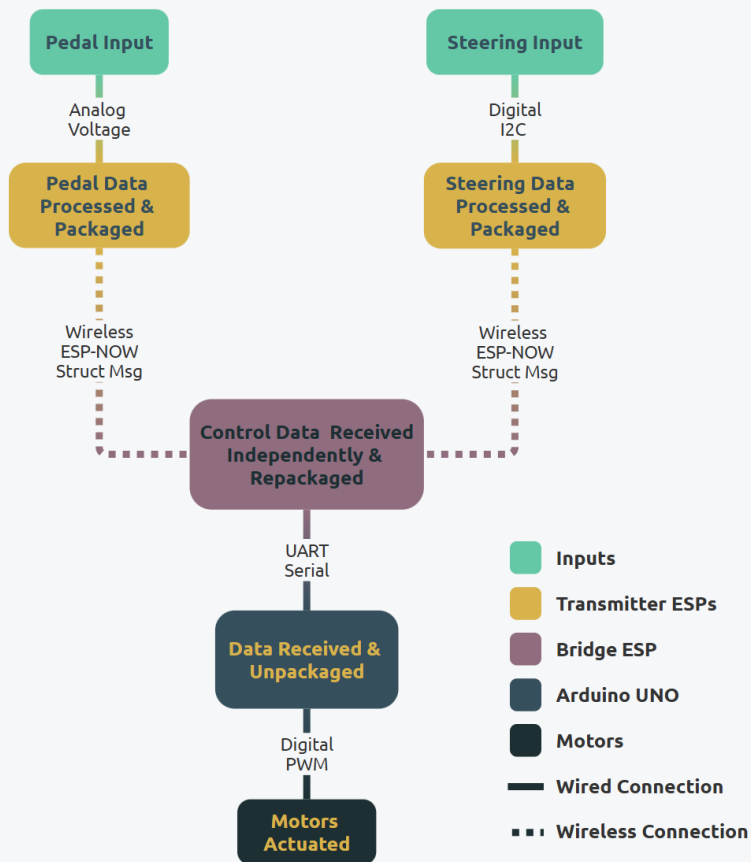


Figure 1: System Architecture Block Diagram

SYSTEM OVERVIEW

The system at a high level consists of five main components which can be seen in figure 1. Utilizing a variety of communication protocols, the system is taking inputs, un-packaging, manipulating, and packaging the data before it sends it wirelessly using ESP-NOW to the onboard rover ESP32 before transferring this data through UART serial communication to the Arduino UNO where it is parsed and turned into PWM signals to actuate the motors. At its core the architecture of the system is divided into three functional stages: Remote Sensing, Wireless Bridging, and Kinetic Actuation.

THE FUNCTIONAL STAGES

1: Remote Sensing and Data Packaging:

The system has two input devices which are hardwired to two transmitting ESP32 nodes. The **Pedal Module** captures raw analog voltage from two potentiometer pedals. This node takes 12-bit ADC values in and maps them to an 8-bit scale (0-255) which can be used by the Arduino to send a PWM signal to actuate the motors. The **Steering Module** uses an MPU6500 to track the rotational orientation of the steering wheel using both the accelerometer and gyroscope data. Which is then filtered to provide a smooth steering output from -90° to 90° . Both nodes then package the data into a structured message and send it through the ESP-NOW protocol to the receiver node.

2: Wireless Communication Bridging:

The receiving ESP32 node acts as the bridge of the two systems. This node is simultaneously receiving two different inputs from the pedal and steering transmission nodes. The node then unpacks the data individually, before packaging the two data sets together. Then sending them via hardware serial connection (UART) to the Arduino motor controller.

3: Motor Control and Actuation:

The final stage is handled by the Arduino UNO with the L293D Motor Shield. The motor control node firstly parses the incoming data from the bridging node to extract the individual control variables. Secondly, the node evaluates the incoming variables by comparing them to a logic tree, in order to send the correct PWM signal to the motor shield to actuate the motors at the right speed in the correct direction.

WIRING SCHEMATICS

The circuit is comprised of three separate modules each communicating with each other. The modules consist of a pedal input module, a free moving steering wheel module, and a rover module. The Pedal Module consists of an Esp32 connected to two 4k potentiometer gas pedals as seen in Fig.(2). The Steering Module consists of an ESP32 connected to an MPU6500 seen in Fig(3). The Driving Module was built from the Lafvin car which we used in Labs 9 and 10 which was modified to communicate with an ESP32 in order to receive the inputs from the pedals and the steering wheel which can be seen in Fig.(4).

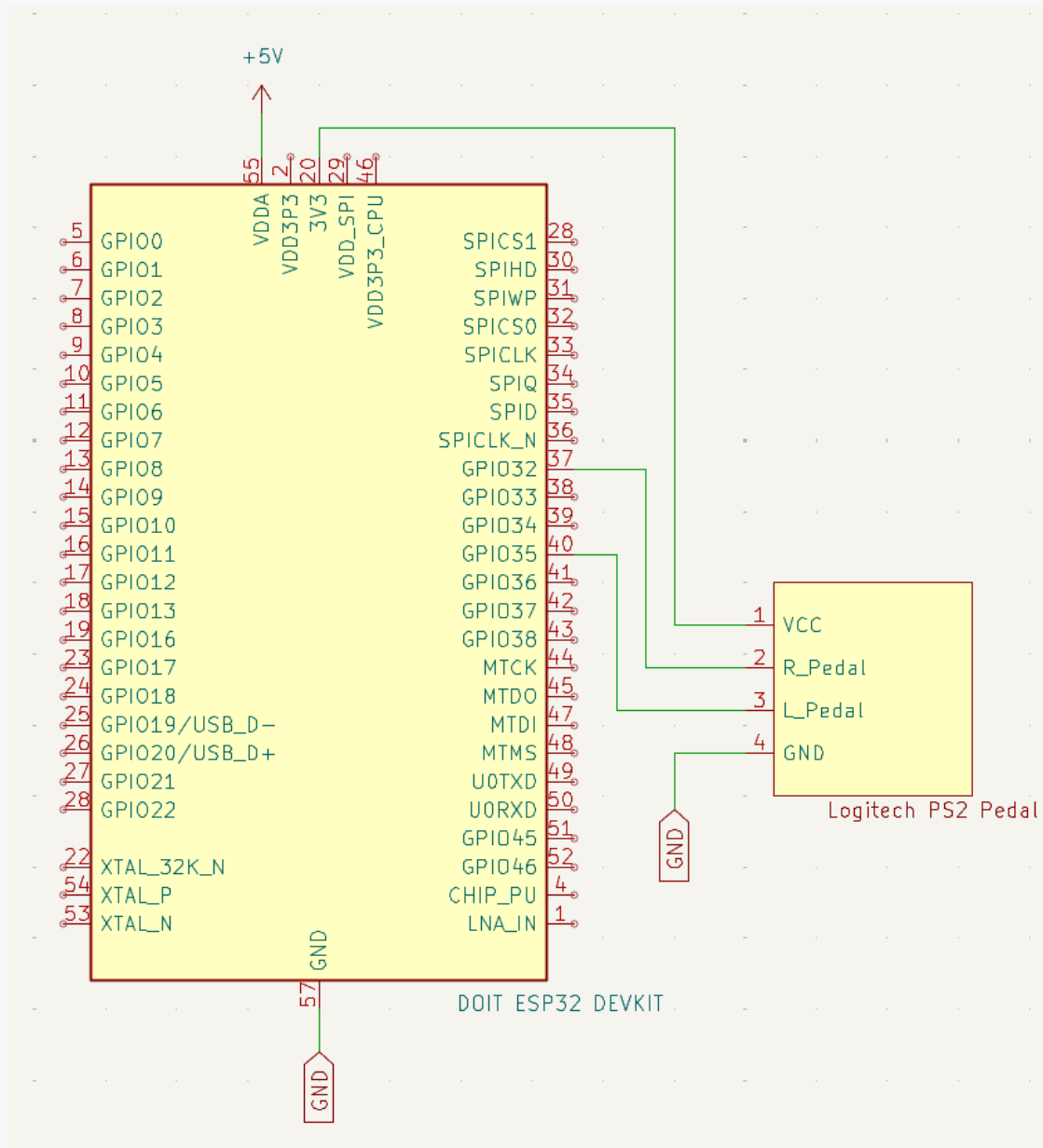


Figure 2: Pedal Module Schematic

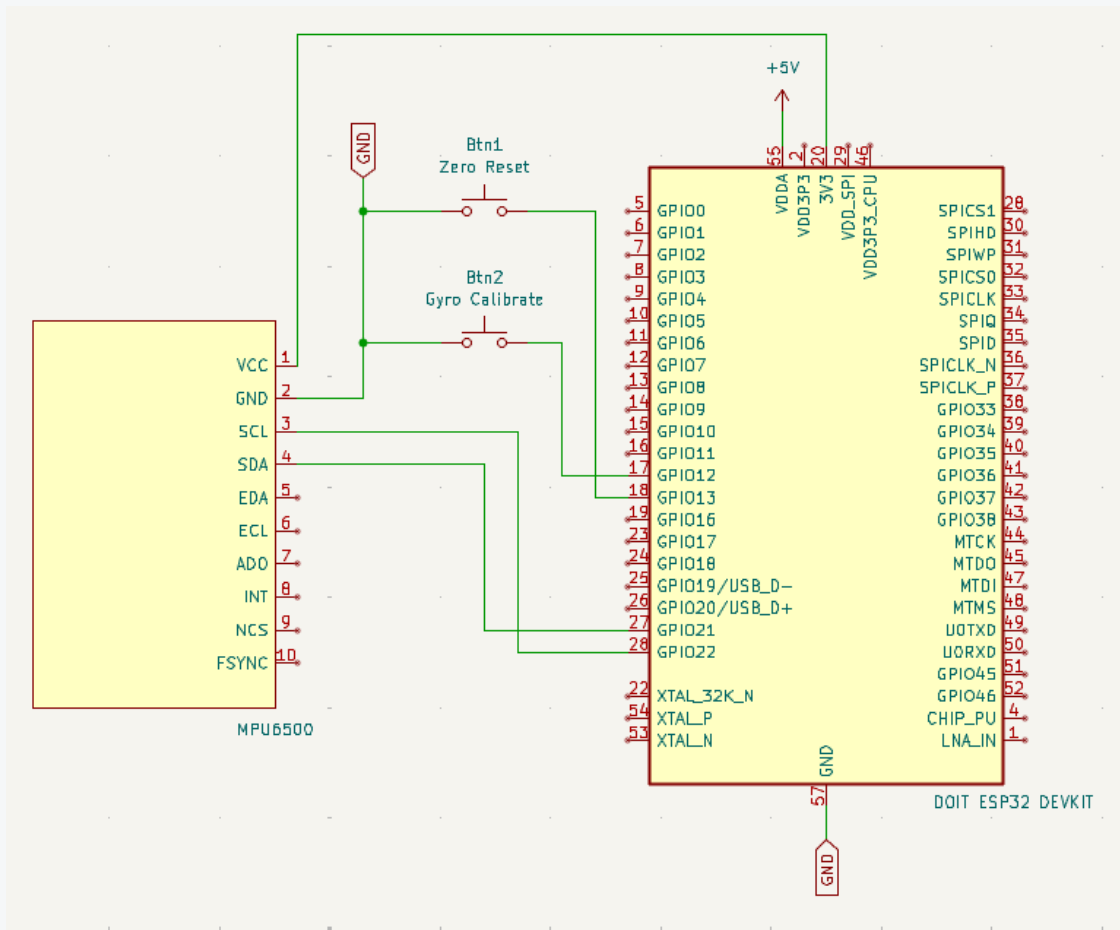


Figure 3: Steering Module Schematic

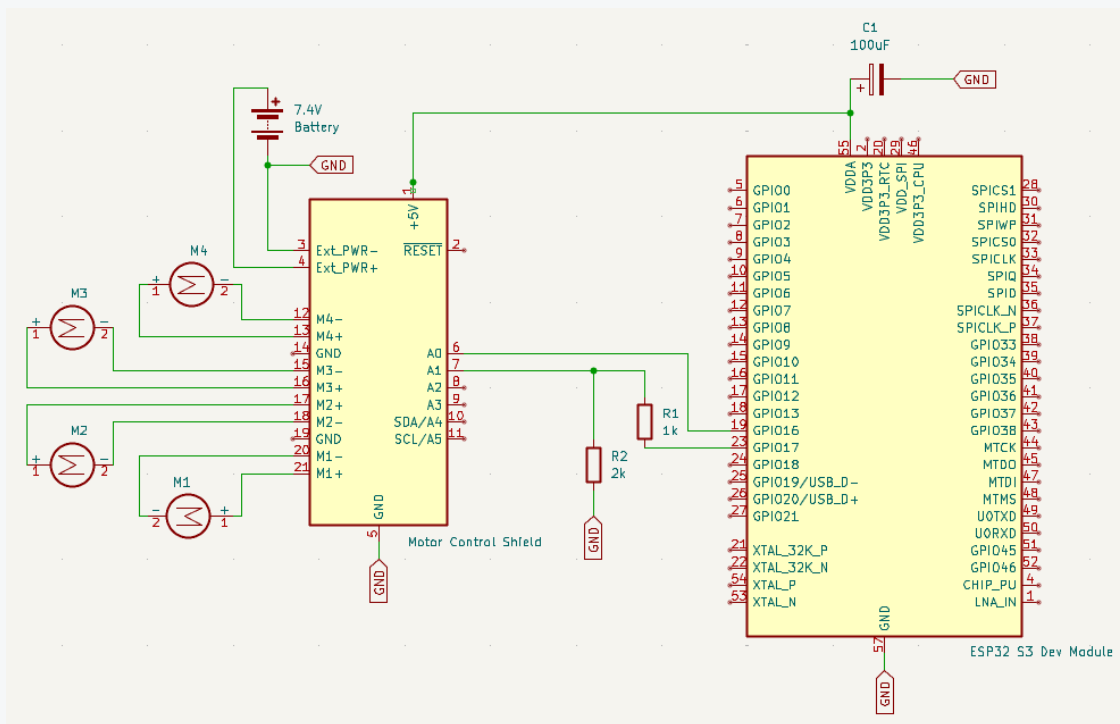


Figure 4: Rover Module Schematic

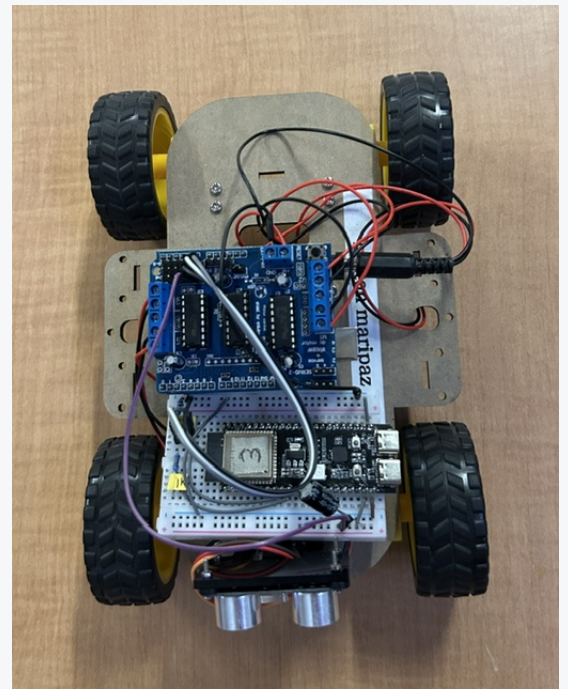
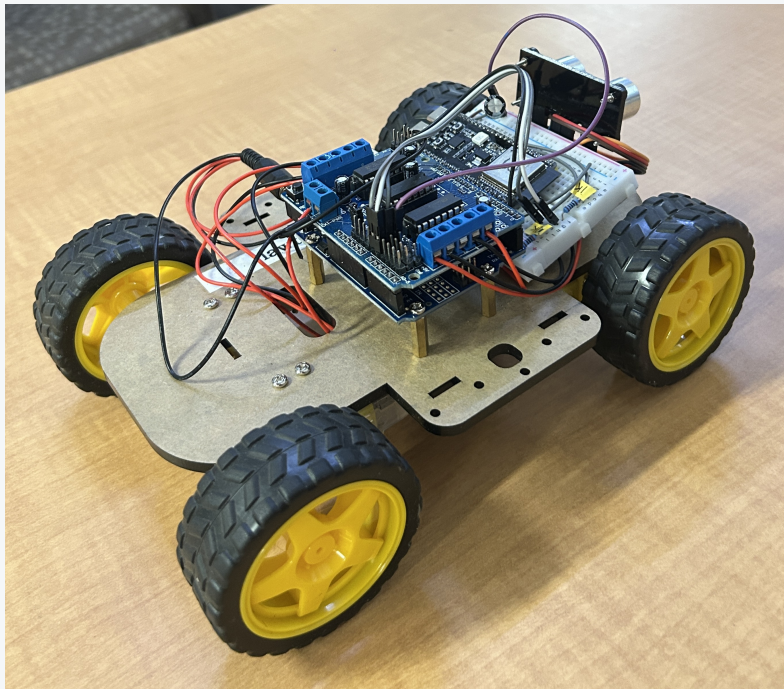


Figure 5: Project CARS Rover Build

STAGE 1: REMOTE SENSING AND DATA PACKAGING

The first stage of Project CARS is the remote sensing and data packaging which consists of the Pedal and Steering modules which is comprised of two input devices and two ESP32 microcontrollers both acting as transmitters to the receiving bridge node. The schematics for these modules can be seen in figures 2 and 3 and the builds can be seen in figures 7 and 9. Both modules are quite simple electrically but provide full control of the Rover.

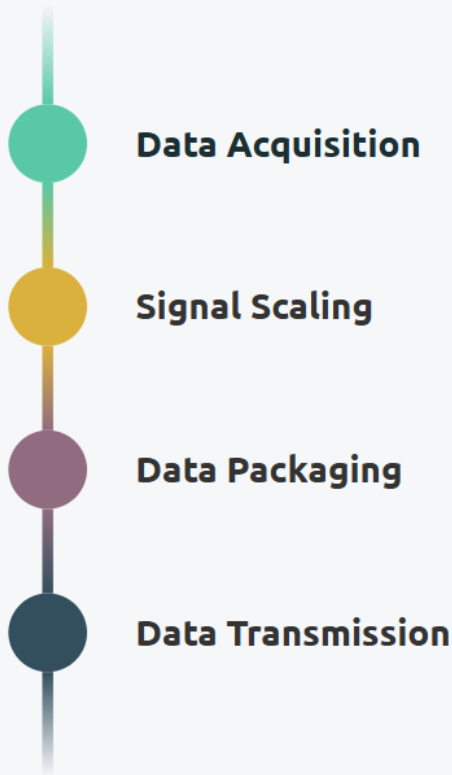


Figure 6: Pedal Module Logic Diagram

This module serves as the primary control interface for movement in the Rover system it can be seen in figure 7. The pedals have four wires: power, ground, and two signal wires. Both the signal wires are connected to 4k potentiometers which are 12-bit ADC signals read by the ESP32. They are then mapped to an 8-bit (0-255) value using `constrain(map(gasVal, 600, 3200, 255, 0), 0, 255);` in order to be handled properly once the data reaches the rover Arduino. The wiring as well as the logic within the pedal module are very simple most of the program consists of ESP-NOW configuration and data sending which is touched on deeper in this report. The operation logic seen below can be followed along in code 1.

Operation Logic:

- Data Acquisition: Reads analog inputs from PIN 32 (Gas) and 35 (Brake).
- Signal Scaling: Maps the specific physical range of the potentiometers (Gas: 600-3200 | Brake: 300-2900) to an 8-bit PWM-ready range (0-255).
- Data Packaging: Packages data into a 'struct_message' containing a transmitter ID ("PEDALS") and the two integer control values.
- Data Transmission: Broadcasts the packet via ESP-NOW at a 20Hz refresh rate (50ms delay) to ensure low-latency rover response.

Code 1: Pedal Controller Data Acquisition Processing & Transmission

```

1 // ACQUISITION- Read the pedal positions
2 int gasVal = analogRead(PIN_GAS_POT); //0-4095 //When pushed down the Pedal is
    analog reading ~490-550 when unpushed reading ~3300-3400
3 int brakeVal = analogRead(PIN_BRAKE_POT); //0-4095 //when pushed down the pedal is
    analog reading ~190-250 when unpushed reading ~2900-3100
4
5 // SCALING- Map Pedal values to 0-255 PWM value for Rover arduino
6 gasVal = constrain(map(gasVal, 600, 3200, 255, 0), 0, 255); //take the inconsistent val from
    gas pedal to a full analog range
7 brakeVal = constrain(map(brakeVal, 300, 2900, 255, 0), 0, 255); //take the inconsistent val
    from gas pedal to a full analog range
8
9 // PACKAGE- Format structured data
10 strcpy(myData.a, "PEDALS"); // Transmitter Code to be understood by Receiver
11 myData.b = gasVal; // Gas Value (0-255)
12 myData.c = brakeVal; // Brake Value (0-255)
13
14 // TRANSMISSION - Send data message via ESP-NOW to receiver
15 esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
16
17 if (result != ESP_OK) {
18     Serial.println("Sending_error");
19 }
20
21 delay(50);

```

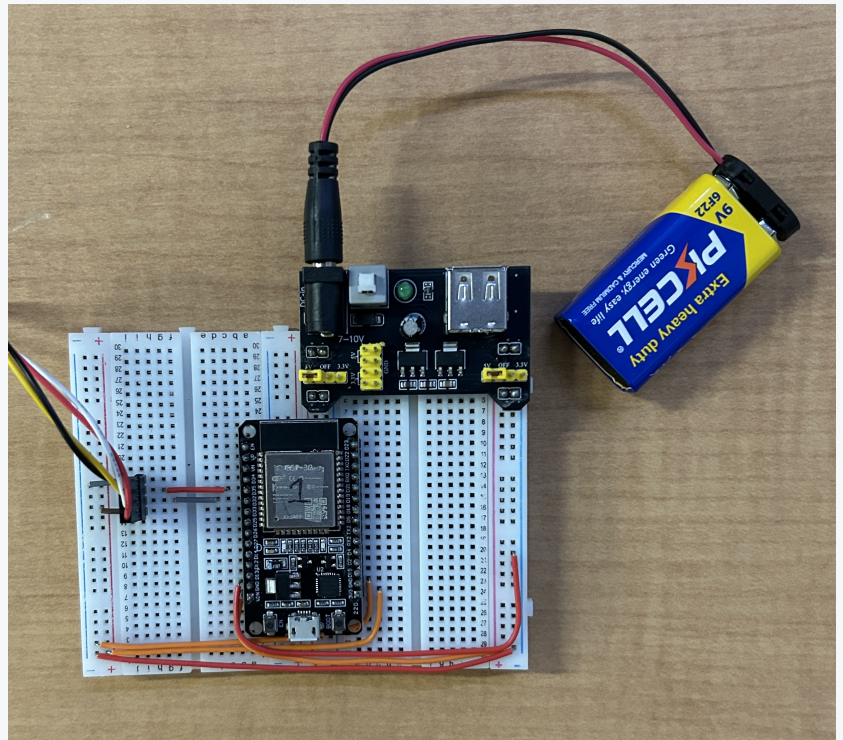
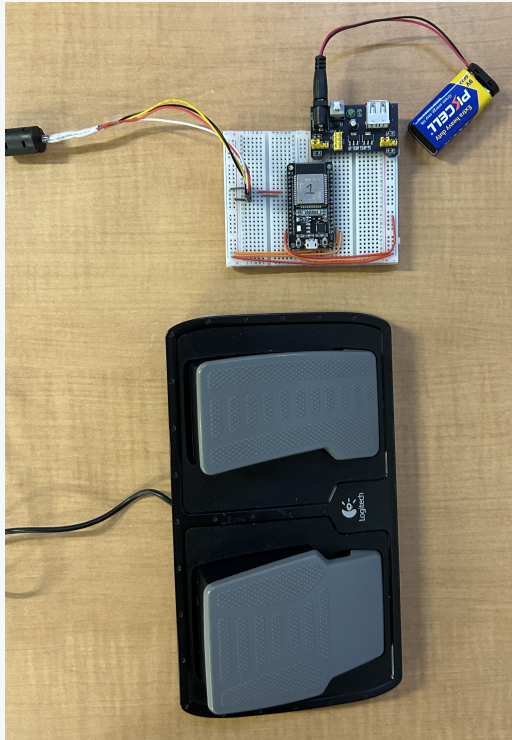


Figure 7: Pedal Module Build

THE STEERING MODULE

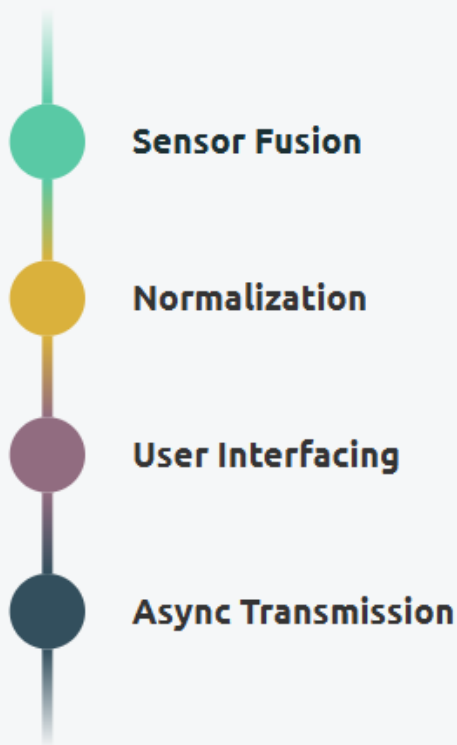


Figure 8: Steering Module Logic Diagram

This module is a 2 breadboard steering wheel that is a wireless control interface using the MPU6500 an Inertial Measurement Unit (IMU) which can be seen in figure 9. It calculates the wheel's rotational orientation through sensor fusion and broadcasts the steering angle to the Rover via the ESP-NOW protocol.

Operation Logic:

- Sensor Fusion: Implements a Complementary Filter combining 98% Gyroscope integration (for fast motion) and 2% Accelerometer data (for long-term stability) to provide a smooth, drift-free angle.
- Normalization: Shifts the coordinate seam to the bottom (180° offset) and constrains the output to a -90° to 90° integer range.
- User Interfacing: Includes physical button interrupts for "Zeroing" (defining the center point) and "Calibration" (calculating Gyro bias).
- Asynchronous Transmission: Utilizes non-interfering millis() timing to send "STEER" data packets at 20Hz without disrupting the filter's dt integration.

SENSOR FUSION

For the steering wheel logic to understand the angle at which the steering wheel was being held at we used the same math and functions as we did in lab 8[3]. Which utilized using $\arctan\left(\frac{A_y}{A_x}\right)$ to calculate the steering angle. However, Project CARS introduced a need for normalization and a filter on the steering wheel in order to prevent accelerometer drift. The normalization was needed as the seam of 0° degrees and 360° was perfectly inline with where the wheel would most often get "zeroed" thus to shift it out of the range of turning; 180° was added to the results of the arc-tangent.

We needed a filter to prevent accelerometer drift. The filter we used can be seen in equation below, where θ is the filtered angle, θ_p is the previous filtered angle, R_g is the gyro rate (which is the gyroscope reading minus it's bias) and α is the accelerometer angle. The gyroscope bias is calculated by taking the average reading when the steering wheel is not moving. This function can be seen in code 2 which is called when the left button on the wheel is pressed.

$$\theta = 0.98(\theta + R_g dt) + 0.02(\alpha) \quad (1)$$

Code 2: Gyroscope Calibration Function

```
1  //----- CALIBRATE GYRO -----
2  // Function to calibrate the gyroscope when it is placed on flat ground
3  // takes the average of 500 gyro readings and assigns it to the global variable gyroBias
4  void calibrateGyro() {
5      long sum = 0;
6      for (int i = 0; i < 500; i++) {
7          mpu.read_gyro();
8          sum += mpu.gz;
9          delay(2);
10     }
11     gyroBias = sum / 500.0;
12 }
```

ASYNCHRONOUS TRANSMISSION

Another important part of the steering logic is the Async transmission in order to maintain a high speed loop for the sensor fusion to not be delayed. We do not need to transmit data or print to the serial monitor every single loop as this adds in unnecessary time delays that affect the change in time for the sensor fusion to combat this we used a non-intrusive timing logic which can be seen in code 3 showing the data transmission.

Code 3: Non-Intrusive Data Transmission Delay

```
1  // Send message via ESP-NOW every 50 ms non interfering (To prevent issues with the gyroscope
   filter integration)
2  if (millis() - lastDelayPrintTime > 50) { //only send every 50ms
3      esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
4      if (result != ESP_OK) {
5          Serial.println("Sending_error");
6      }
7      lastDelayPrintTime = millis(); //reset last time of data sent
8  }
```

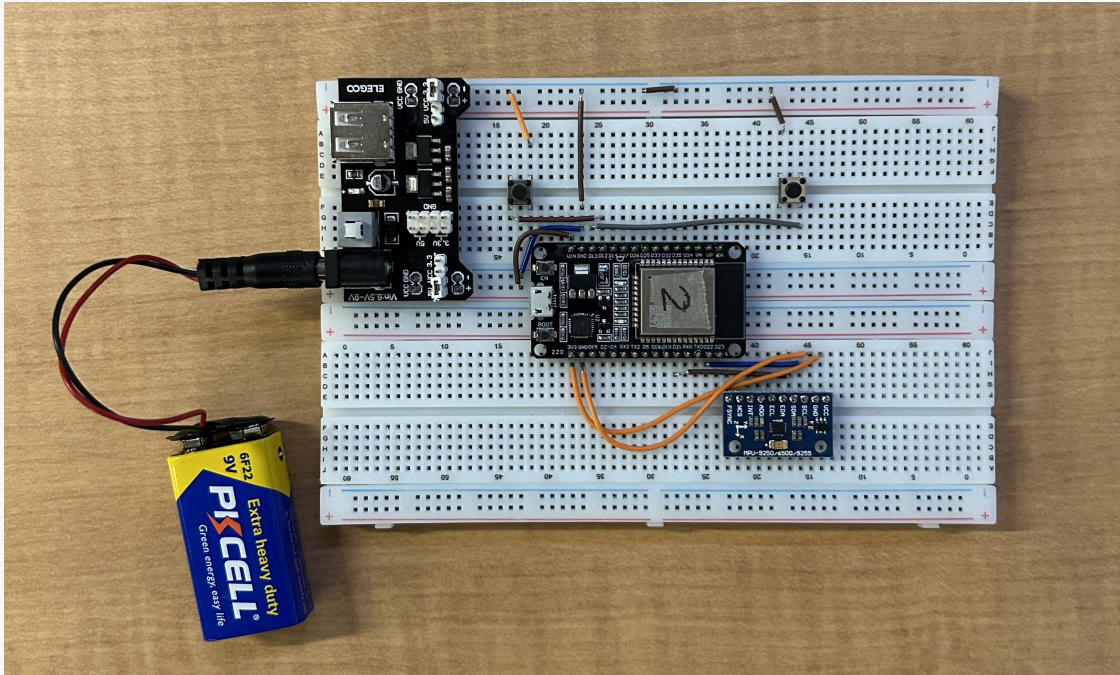


Figure 9: Steering Module Build

STAGE 2: COMMUNICATION BRIDGE



This module acts as the central communication hub for the Rover. It receives wireless control packets from multiple transmitters via ESP-NOW, aggregates the inputs (Gas, Brake, and Steering), and relays them to the primary motor control unit (Arduino) via a packetized UART stream.

Operation Logic:

- **Wireless Gateway:** Listens for ESP-NOW traffic and utilizes a callback function to differentiate between "PEDALS" and "STEER" data sources.
- **Data Synchronization:** Stores the most recent 8-bit pedal values and steering angles into global variables for asynchronous processing.
- **Serial Packaging:** Encapsulates the control data into a delimited packet format: "<rPedal,lPedal,steerAngle>" to ensure data integrity.
- **Hardware Bridge:** Transmits the formatted string over HardwareSerial1 (UART1) at 9600 Baud to the downstream Arduino every 50ms.

Figure 10: Communication Bridge Logic Diagram

COMMUNICATION BRIDGE AND SIGNAL INTEGRATION

The central communication hub of Project C.A.R.S. utilizes a dual-layered protocol to bridge wireless user inputs to the physical motor controllers. This system acts as a gateway, translating high-speed wireless packets into a synchronized serial stream.

WIRELESS PROTOCOL: ESP-NOW

To minimize latency in rover operation, the ESP-NOW protocol was implemented. Unlike standard Wi-Fi, ESP-NOW is a connectionless, low-power protocol that allows for peer-to-peer communication between microcontrollers without the need for a central router. This ensures near-instantaneous transmission of control data from the steering and pedal units to the rover's receiver. To use ESP-NOW we used the `esp_now.h` library which requires initializing and registering the data call back using `esp_now_init()` and `esp_now_register_recv_cb(OnDataRecv)`;

DATA STRUCTURE AND ENCAPSULATION

To ensure data integrity across the wireless link, a uniform data structure, `struct_message`, was defined. This structure was consistent across all the ESP nodes, which allowed the system to encapsulate multiple data types into a single packet, which is then mapped directly in memory upon arrival.

Code 4: ESP-NOW Data Structure

```
1 typedef struct struct_message {
2     char a[32]; // Identifier: "PEDALS" or "STEER"
3     int b;      // Primary value (Gas or Steering Angle)
4     int c;      // Secondary value (Brake)
5 } struct_message;
```

The `char a[32]` array acts as a header or "ID tag," allowing the receiver to differentiate between multiple transmitters on the same frequency.

ASYNCHRONOUS RECEIVING AND CALLBACK LOGIC

The receiver utilizes a callback function, `OnDataRecv`, which is triggered automatically by the ESP32 hardware whenever a packet is detected. This prevents the main loop from stalling while waiting for data. Within this function, the `memcpy()` command is used to copy the incoming byte stream into the `myData` object.

The logic then uses string comparison (`strcmp`) to route the data:

- **Pedal Data:** Updates the global `gasVal` and `brakeVal` (scaled 0-255).
- **Steering Data:** Updates the global `steerVal` (scaled -90 to 90 degrees).

Once the data is aggregated, it is relayed to the downstream Arduino motor controller via Universal Asynchronous Receiver-Transmitter (UART) communication.

Code 5: Callback Logic

```
1 // Callback function executed when data is received
2 void OnDataRecv(const esp_now_recv_info_t * recv_info, const uint8_t *incomingData, int len) {
3     memcpy(&myData, incomingData, sizeof(myData));
4
5     //based on data recieved we set the gas, brake and steering values to be sent to the arduino
6
7     //DATA FROM PEDALS
8     if(strcmp(myData.a, "PEDALS") == 0){ // Check for message from pedal transmitter
9         gasVal = myData.b; // GasPedal Value from the pedals (0,255) range
10        brakeVal = myData.c; // Brake Pedal value from pedals (0,255) range
11    }
12    //DATA FROM STEERING
13    else if(strcmp(myData.a, "STEER") == 0){ //check for message from steering transmitter
14        steerVal = myData.b; //Steering Value from Steering unit (-90, 90) range
15    }
```

SERIAL PACKAGING

To transmit the data from the ESP32 to the Arduino to control the motor we used the Hardware Serial library in order to remap the serial communication of the ESP to pins 16 and 17 which can be seen in figure 4. Data is transmitted over `HardwareSerial1` at a baud rate of 9600. To ensure the Arduino can reliably parse the data, the values are encapsulated in a packetized string format: `<rPedal,lPedal,steerAngle>`. The use of `<` and `>` characters as start and end markers allows the Arduino to ignore line noise and only process complete packets.

LOGIC LEVEL SHIFTING AND VOLTAGE DIVIDER

A critical hardware constraint in this bridge is the voltage discrepancy between the two microcontrollers. The ESP32 operates on 3.3V logic, while the Arduino operates on 5V logic. Connecting the Arduino's Transmit (TX) pin directly to the ESP32's Receive (RX) pin could cause overvoltage damage to the ESP32.

To safely interface these devices, a **voltage divider** is implemented on the Arduino's TX line. By using two resistors, the 5V signal is stepped down to a safe 3.3V level. The output voltage is calculated using the following relationship:

$$V_{out} = V_{in} \cdot \left(\frac{R_2}{R_1 + R_2} \right)$$

For this project, a $1k\Omega$ resistor (R_1) and a $2k\Omega$ resistor (R_2) were utilized to achieve a theoretical V_{out} of approximately 3.33V, ensuring signal compatibility and hardware longevity. This divider can be seen in the schematic figure 11 and figure 5 to left of the ESP labeled "3".

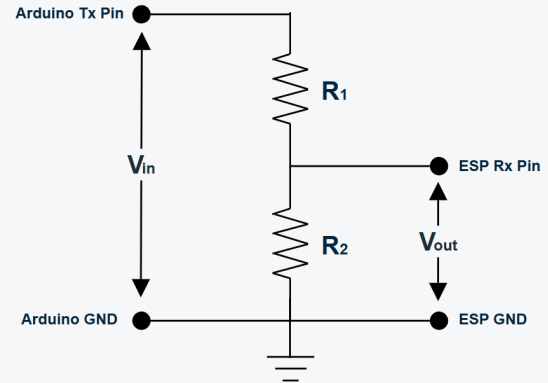
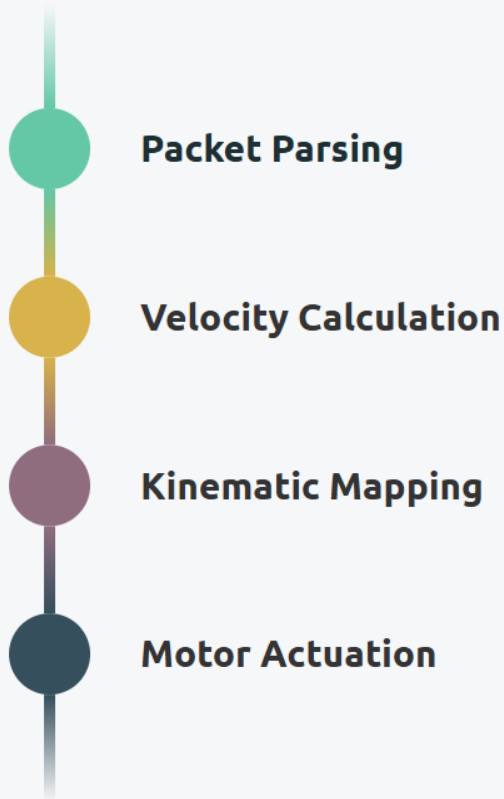


Figure 11: Serial Communication Voltage Divider

STAGE 3: MOTOR CONTROL AND ACTUATION



This module serves as the primary power and movement controller for the rover. It decodes the telemetry packets received from the ESP32 serial bridge and translates those control signals into physical movement via an L293D motor driver shield and a four-motor drivetrain.

Operation Logic:

- Packet Parsing: Listens on `SoftwareSerial` for the `'<'` start delimiter and parses the CSV-formatted gas, brake, and steering telemetry.
- Velocity Calculation: Derives a net `'speedVal'` by subtracting brake input from gas input to determine forward/reverse magnitude.
- Kinematic Mapping: Implements a three-tier steering logic:
 - Deadzone ($\pm 10^\circ$): Maintains synchronized RPM for straight travel.
 - Differential Turn ($10^\circ - 50^\circ$): Scales inner wheel speed relative to the steering angle for smooth arc turns.
 - Pivot/Rotate ($> 50^\circ$): Executes zero-radius tank turns by reversing the rotation of inner wheels.
- Interfaces with an 8-bit shift register to set H-bridge directions and applies PWM signals to regulate motor torque.

Figure 12: Motor Control Logic Diagram

STEERING CONTROL LOGIC AND INPUT MAPPING

The steering system serves as the primary directional interface for the rover, converting mechanical rotation into a digital signal. In order to actuate the motor we used the same logic as found in Lab 9[4] utilizing a custom function to actuate each motor in the desired direction and speed. In order to have a smooth steering feeling dynamic steering was utilized, which was obtained by varying the speed being passed to the inner and outer motors.

As illustrated in Figure 14, the steering wheel is divided into angular zones to provide a different turning response based on user input. The control logic for the rover follows a simple if-else tree actuating the motors based on the driving zone you are in which is calculated from your movement `speedVal` and steering `steerAngle` values. Figure 13 displays the logic flow of the driving control the full implemented control logic can be found in Appendix A.4.

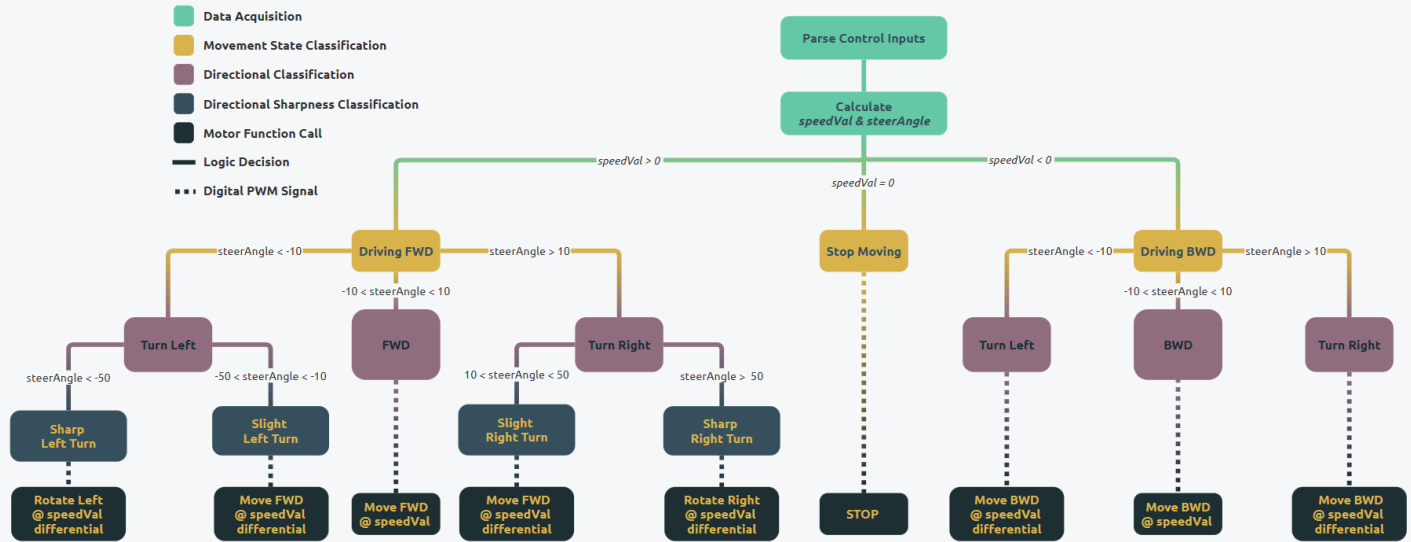


Figure 13: Driving Logic Diagram

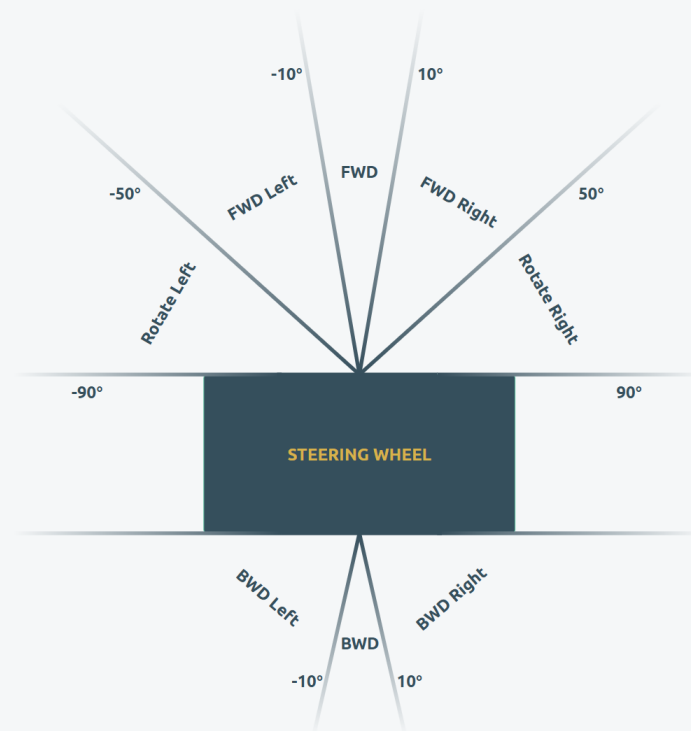


Figure 14: Steering Angle Diagram

ARDUINO INTEGRATION VIA SOFTWARE SERIAL

While the ESP32 receiver utilizes its dedicated `HardwareSerial` for high-speed transmission, the downstream Arduino which acting as the motor controller utilizes the `SoftwareSerial` library. This configuration is necessary to reserve the Arduino's hardware UART (Pins 0 and 1) for PC-based debugging and flashing via the USB interface.

Interface Configuration

The communication bridge is established by defining virtual RX and TX pins on the Arduino. To ensure data synchronization with the 50ms refresh rate of the ESP32, the `SoftwareSerial` buffer must be handled with a non-blocking read logic to parse the incoming `<rPedal,lPedal,steerAngle>` packet. The logic for establishing the Software Serial can be seen in code 6

Code 6: Software Serial Setup

```
1  #include <SoftwareSerial.h>
2
3  //init Serial at analog pins
4  #define RxPIN    A0
5  #define TxPIN    A1
6
7  //Set up Software Serial Monitor for ESP communication with new pins
8  SoftwareSerial mySerial(RxPIN, TxPIN); // RX, TX
9
10 void setup() {
11     Serial.begin(9600); // begin serial monitor for computer communication
12     mySerial.begin(9600); // begin the serial communication between the ESP and Arduino (
        Software Serial)
13 }
```

Parsing Serial Data

In order to parse the data that has been packaged from the ESP Receiver node. Firstly, we read from the software serial monitor if something is available. Secondly, we check if the start delimiter "<" is recognized, if so then the three data values are parsed and stored in their respective variables as seen in code 7.

Code 7: Software Serial Data Parsing

```
1  if (mySerial.available() > 0) {
2      char startChar = mySerial.read(); //read serial data from esp
3
4      // Check if this is the start of our data packet
5      // if the data is formatted in the correct form read the driving control variables from
        esp
6      if (startChar == '<') {
7          gasVal = mySerial.parseInt(); // Reads up to the first comma,
8          brakeVal = mySerial.parseInt(); // Reads up to the second comma
9          steerAngle = mySerial.parseInt(); // Reads up to the '>'
10     }
11 }
```

Signal Integrity and Logic Shifting

Because the Arduino operates on 5V logic, the Tx signal (Pin A1) cannot be connected directly to the ESP32 RX pin. As previously discussed in the Hardware Bridge section, a voltage divider is placed at this junction. Conversely, the 3.3V TX signal from the ESP32 is sufficient to trigger the "High" logic state on the Arduino's 5V RX pin (Pin A0) without additional amplification.

PROJECT MANAGEMENT

TIMELINE

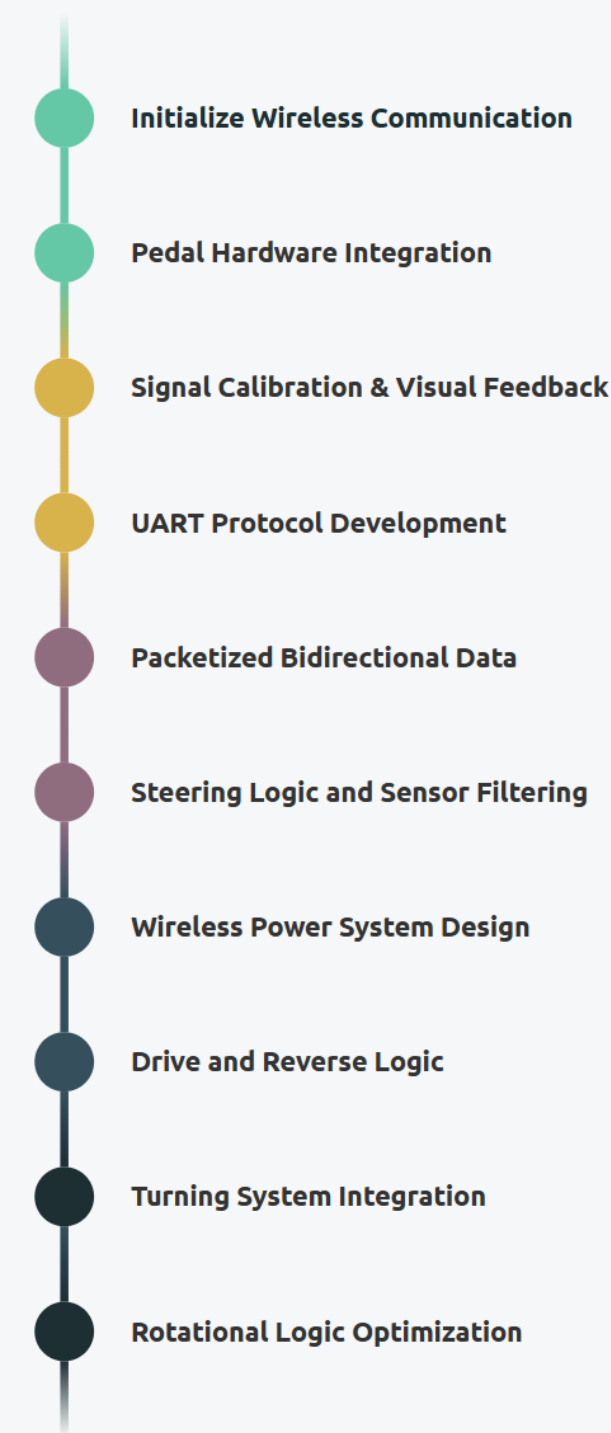


Figure 15: Project Timeline

Below is a high level overview of the steps taken during Project CARS.

- **Initialize Wireless Communications:** Set-up ESP-NOW and identify ESP32 MAC Addresses.
- **Pedal Hardware Integration:** Soldered pedal wiring for stable breadboard connections.
- **Signal Calibration & Visual Feedback:** Defined pedal ranges and mapped output to LED indicators
- **UART Protocol Development:** Established serial communication between ESP32 and Arduino
- **Packetized Bidirectional Data:** Implemented structured data exchange for pedal and steering inputs.
- **Steering Logic & Sensor Filtering:** Processed accelerometer and gyro data with constraints for wheel positioning.
- **Wireless Power System Design:** Configured external power with decoupling capacitors and common grounding.
- **Drive & Reverse Logic:** Programmed motor controls to handle acceleration and braking transitions.
- **Turning System Integration:** Developed standard directional control logic.
- **Rotational Logic Optimization:** Implemented advanced sharper turning and rotating maneuvers.

INITIAL PLANNED TIMELINE

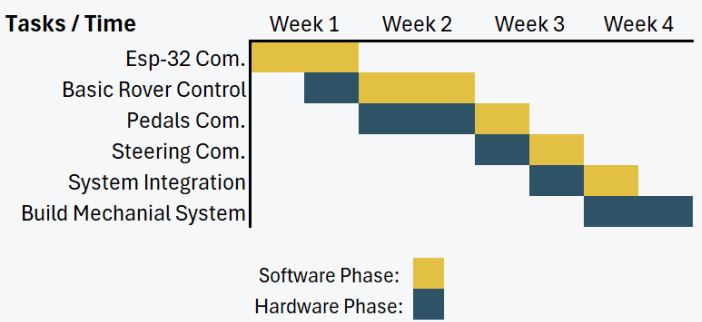


Figure 16: Initial Planned Timeline

CONCLUSION

RESULTS

Project CARS successfully met 4 of the 5 design requirements falling short on the fail safe requirement due to time constraints, however, the project resulted in a wireless controlled rover with a three tiered kinematic mapping system to create a fully dynamic operation scheme. The system provided a 10° deadzone for straight travel, differential turns between $10^\circ - 50^\circ$ for smooth arcs and tight radius turns for steering angles greater than 50° . The control system implemented sensor fusion for the IMU based steering wheel through use of a complimentary filter of 98% gyroscope and 2% accelerometer data, this successfully allowed the steering to stay straight with no sensor drift. All whilst successfully packaging and parsing data through the wireless ESP-NOW protocol.

FINAL REFLECTION

In conclusion, Project C.A.R.S. (Control and Actuation via Remote Systems) was a culmination of everything we have learned this year and more. Through this project, I have found extensive learning in wireless and serial communication, hardware interfacing, and control systems implementation. By successfully bridging high-speed wireless packets via the ESP-NOW protocol with UART serial streams, the system demonstrated an architecture applicable to real-time remote actuation. The integration of diverse and unknown hardware, provided practical experience in signal scaling, logic level shifting, and managing hardware constraints. Ultimately, the successful implementation of a multi-node wireless system and mostly successful completion of the design requirements proved that specialized microcontrollers can be used to effectively execute complex mechanical tasks at high-speed.

References

- [1] Esp32 UART0, UART1, UART2 access using the Arduino Ide (no date) Copperhill. Available at: <https://copperhilltech.com/blog/esp32-serial-ports-uart0-uart1-uart2-access-using-the-arduino-ide/> (Accessed: 05 May 2026).
- [2] DroneBot Workshop (2023) ESP now - peer to peer ESP32 communications, DroneBot Workshop. Available at: <https://dronebotworkshop.com/esp-now/> (Accessed: 05 May 2026).
- [3] Jones, L. (2026, March 23). Microcontroller systems lab 8: IMUs: Accelerometer and gyroscopes. University of Vermont, Department of Mechanical Engineering.
- [4] Jones, L. (2026, March 30). Microcontroller systems lab 9: IR remotes & cars. University of Vermont, Department of Mechanical Engineering.

APPENDIX

MICROCONTROLLER SOURCE CODES

PEDAL CONTROLLER TRANSMITTER

Code 8: Pedal Controller Transmitter

```
1  /*
2  * -----
3  * PROJECT: Project C.A.R.S. | Wireless Rover Control
4  * MODULE:  Pedal Controller Transmitter
5  * AUTHOR:  Lawton Jones
6  * COLLEGE: University of Vermont | Dept. of Mechanical Engineering
7  * COURSE:  CMPE 3815: Microcontrollers
8  * DATE:    4 May 2026
9  * -----
10 * * DESCRIPTION:
11 * This script serves as the primary control interface for the Rover system.
12 * It reads analog voltage signals from two Logitech PS2 potentiometer pedals
13 * (Gas and Brake), processes the raw 12-bit ADC data, and transmits the
14 * normalized values to the receiver ESP32 via the ESP-NOW protocol.
15 * * OPERATION LOGIC:
16 * 1. Data Acquisition: Reads analog inputs from PIN 32 (Gas) and 35 (Brake).
17 * 2. Signal Scaling: Maps the specific physical range of the potentiometers
18 * (Gas: 600-3200 | Brake: 300-2900) to an 8-bit PWM-ready range (0-255).
19 * 3. Encapsulation: Packages data into a 'struct_message' containing a
20 * transmitter ID ("PEDALS") and the two integer control values.
21 * 4. Transmission: Broadcasts the packet via ESP-NOW at a 20Hz refresh rate
22 * (50ms delay) to ensure low-latency rover response.
23 * * HARDWARE CONFIGURATION:
24 * - MCU: ESP32 (Transmitter)
25 * - Input: Logitech Pedal Assembly
26 * - Power: 5V Power Supply Module (common ground with ESP32)
27 * - Pin 32: Gas Potentiometer (Analog In)
28 * - Pin 35: Brake Potentiometer (Analog In)
29 * -----
30 */
31
32
33
34
35 //----- LIBRARIES -----
36 #include <esp_now.h>
37 #include <WiFi.h>
38
39
40
41 //----- INIT -----
42 //define Gas and Brake pedal input pins
43 const byte PIN_GAS_POT = 32;    //right gas pedal potentiometer pin
44 const byte PIN_BRAKE_POT = 35;  //right gas pedal potentiometer pin
45
46
47
48 // MAC Address of responder - edit as required
49 uint8_t broadcastAddress[] = {0x3C, 0xDC, 0x75, 0x6E, 0x90, 0x24}; //mac address of the
    receiver (Rover ESP) (3)
50
51 //Define the Data structure that all ESP's recognize
52 // char is the Transmitter Code to be recognized by receiver
53 // b and c are gas and brake values
```

```

54 typedef struct struct_message {
55     char a[32];
56     int b;
57     int c;
58 } struct_message;
59
60 // Create a structured object
61 struct_message myData;
62
63 // Peer info for ESP communication
64 esp_now_peer_info_t peerInfo;
65
66 // Callback function called when data is sent
67 // used to debug and confirm ESP is properly sending data
68 void OnDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
69     Serial.print("\r\nLast_Packet_Send_Status:\t");
70     Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery_Success" : "Delivery_Fail");
71 }
72
73
74
75
76
77
78 //----- SETUP -----
79 void setup() {
80
81     //init pedal pins
82     pinMode(PIN_GAS_POT, INPUT);
83     pinMode(PIN_BRAKE_POT, INPUT);
84
85     // Set up Serial Monitor
86     Serial.begin(115200);
87
88     // Set ESP32 as a Wi-Fi Station
89     WiFi.mode(WIFI_STA);
90
91     // Initilize ESP-NOW
92     if (esp_now_init() != ESP_OK) {
93         Serial.println("Error_initializing_ESP-NOW");
94         return;
95     }
96
97     // Register the send callback
98     esp_now_register_send_cb(OnDataSent);
99
100    // Register peer
101    memcpy(peerInfo.peer_addr, broadcastAddress, 6);
102    peerInfo.channel = 0;
103    peerInfo.encrypt = false;
104
105    // Add peer
106    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
107        Serial.println("Failed_to_add_peer");
108        return;
109    }
110 }
111
112
113
114
115
116 //----- LOOP -----
117 void loop() {

```

```

118
119
120 // Read the pedal positions
121 int gasVal = analogRead(PIN_GAS_POT); //0-4095 //When pushed down the Pedal is
    analog reading ~490-550 when unpushed reading ~3300-3400
122 int brakeVal = analogRead(PIN_BRAKE_POT); //0-4095 //when pushed down the pedal is
    anlaog reading ~190-250 when unpushed reading ~2900-3100
123
124 // Map Pedal values to 0-255 PWM value for Rover arduino
125 gasVal = constrain(map(gasVal,600,3200,255,0), 0, 255); //take the inconsistent val from
    gas pedal to a full analog range
126 brakeVal = constrain(map(brakeVal,300,2900,255,0), 0, 255); //take the inconsistent val
    from gas pedal to a full analog range
127
128
129 // Format structured data
130 strcpy(myData.a, "PEDALS"); // Transmitter Code to be understood by Receiver
131 myData.b = gasVal; // Gas Value (0-255)
132 myData.c = brakeVal; // Brake Value (0-255)
133
134 // Send data message via ESP-NOW to receiver
135 esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
136
137 if (result != ESP_OK) {
138     Serial.println("Sending_error");
139 }
140
141
142 delay(50); //Wait 50 ms so the data is sent at a rate that can be handled by ESP and
    Arudino
143 }
144
145
146
147
148 //----- END OF SKETCH -----
149 //----- END OF SKETCH -----
150 //----- END OF SKETCH -----

```

Code 9: Steering Wheel Transmitter

```
1  /*
2   * -----
3   * PROJECT: Project C.A.R.S. | Wireless Rover Control
4   * MODULE:  Steering Wheel Transmitter (IMU-based)
5   * AUTHOR:  Lawton Jones
6   * COLLEGE: University of Vermont | Dept. of Mechanical Engineering
7   * COURSE:  CMPE 3815: Microcontrollers
8   * DATE:    4 May 2026
9   * -----
10  * * DESCRIPTION:
11  * This script transforms a 2 breadboard steering wheel into a wireless control
12  * interface using an Inertial Measurement Unit (IMU). It calculates the
13  * wheel's rotational orientation through sensor fusion and broadcasts the
14  * steering angle to the Rover via the ESP-NOW protocol.
15  * * OPERATION LOGIC:
16  * 1. Sensor Fusion: Implements a Complementary Filter combining 98% Gyroscope
17  * integration (for fast motion) and 2% Accelerometer data (for long-term
18  * stability) to provide a smooth, drift-free angle.
19  * 2. Normalization: Shifts the coordinate seam to the bottom (180 offset)
20  * and constrains the output to a -90 to 90 integer range.
21  * 3. User Interfacing: Includes physical button interrupts for "Zeroing"
22  * (defining the center point) and "Calibration" (calculating Gyro bias).
23  * 4. Async Transmission: Utilizes non-interfering millis() timing to send
24  * "STEER" data packets at 20Hz without disrupting the filter's dt integration.
25  * * HARDWARE CONFIGURATION:
26  * - MCU: ESP32 (Transmitter)
27  * - Sensor: MPU6500 (9-Axis IMU) via I2C (SDA: 21, SCL: 22)
28  * - Inputs: Reset/Zero Button (Pin 13), Calibration Button (Pin 12)
29  * - Power: 5V Power Supply Module (common ground with ESP32)
30  * - Communications: ESP-NOW (Peer-to-Peer Wireless)
31  * -----
32  * SOURCES:
33  * - https://dronebotworkshop.com -iESP-NOW Demo - Transmit
34  *
35  *
36  */
37
38
39 //----- LIBRARIES -----
40 // Include Libraries
41 #include <esp_now.h>
42 #include <WiFi.h>
43 #include <MPU9255.h> //include MPU9255 library
44
45
46
47
48
49 //----- INIT -----
50 MPU9255 mpu; //init accelerometer
51
52
53
54 //define the acclerometer and I2C pins
55 #define PIN_RESET_BTN 13 //right button to reset the steering angle
56 #define PIN_CALIBRATE_BTN 12 //left button to calibrate the gyroscope on a flat surface.
57 #define SDA 21
58 #define SCL 22
59
```

```

60
61 //Intialize the variables for calculations Angle of steering wheel
62 float zeroAngle = 0;           //var that holds current angle of system when zeroed
63 float filteredAngle = 0;       // var that holds angle of steady angle based on gyro damping
64 float gyroBias = 0;           // Calibration offset
65
66 // define variables for timed printing and data sending
67 unsigned long lastPrintTime = 0;
68 unsigned long lastTime = 0;
69 unsigned long lastDelayPrintTime = 0;
70
71 // MAC Address of responder - mac address of the receiver
72 uint8_t broadcastAddress[] = {0x3C, 0xDC, 0x75, 0x6E, 0x90, 0x24}; //mac address of the
    receiver (3)
73
74 // Define a data structure **Must be consistent between all communicating ESPs
75 // char is the Transmitter Code to be recognized by receiver
76 // b is the steering angle value
77 typedef struct struct_message {
78     char a[32];
79     int b;
80     int c;
81 } struct_message;
82
83 // Create a structured object
84 struct_message myData;
85
86 // Peer info
87 esp_now_peer_info_t peerInfo;
88
89 // Callback function called when data is sent
90 // Change "const uint8_t *mac_addr" to "const wifi_tx_info_t *info"
91 void OnDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
92     //Serial.print("\r\nLast Packet Send Status:\t");
93     //Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery Fail");
94 }
95
96
97
98
99
100 //----- FUNCTIONS -----
101
102 //----- CALIBRATE GYRO -----
103 // Function to calibrate the gyroscope when it is placed on flat ground
104 // takes the average of 500 gyro readings and assigns it to the global variable gyroBias
105 void calibrateGyro() {
106     long sum = 0;
107     for (int i = 0; i < 500; i++) {
108         mpu.read_gyro();
109         sum += mpu.gz;
110         delay(2);
111     }
112     gyroBias = sum / 500.0;
113 }
114
115
116
117
118
119
120
121
122 //----- SETUP -----

```

```

123 void setup() {
124     // Set up Serial Monitor
125     Serial.begin(115200);
126
127     // begin I2C protocol on pins 16 and 17
128     Wire.begin(SDA, SCL);
129
130     //MPU initialization check
131     if(mpu.init()){
132         Serial.println("initialization_Successful");
133     } else{
134         Serial.println("initialization_Failed");
135     }
136
137     //set the scales and bandwidth for the Accelerometer and Gyroscope
138     mpu.set_acc_scale(scale_2g); //2g, 4g, 8g, 16g
139     mpu.set_gyro_scale(scale_500dps); //500 degrees/sec
140     mpu.set_acc_bandwidth(acc_20Hz);
141     mpu.set_gyro_bandwidth(gyro_20Hz);
142
143     //Set button pins to inputs with pullup resistors
144     pinMode(PIN_RESET_BTN, INPUT_PULLUP);
145     pinMode(PIN_CALIBRATE_BTN, INPUT_PULLUP);
146
147     // Set ESP32 as a Wi-Fi Station
148     WiFi.mode(WIFI_STA);
149
150     // Initilize ESP-NOW
151     if (esp_now_init() != ESP_OK) {
152         Serial.println("Error_initializing_ESP-NOW");
153         return;
154     }
155
156     // Register the send callback
157     esp_now_register_send_cb(OnDataSent);
158
159     // Register peer
160     memcpy(peerInfo.peer_addr, broadcastAddress, 6);
161     peerInfo.channel = 0;
162     peerInfo.encrypt = false;
163
164     // Add peer
165     if (esp_now_add_peer(&peerInfo) != ESP_OK){
166         Serial.println("Failed_to_add_peer");
167         return;
168     }
169 }
170
171
172
173
174
175
176 //----- LOOP -----
177 void loop() {
178
179     // Timing for Integration
180     unsigned long currentTime = millis();
181     float dt = (currentTime - lastTime) / 1000.0; // Change in time in seconds
182     lastTime = currentTime;
183
184     // read data from the MPU6500
185     mpu.read_acc(); //get data from the accelerometer
186     mpu.read_gyro(); //get data from the gyroscope

```

```

187
188 // Assign raw data values from MPU to arrays of their X,Y,Z values
189 float Accelerometer_values[3] = {mpu.ax, mpu.ay, mpu.az};
190 int gyroscope_values[3] = {mpu.gx, mpu.gy, mpu.gz};
191
192 //Convert the raw data to values of G
193 for (byte I = 0; I < 3; I++){
194     Accelerometer_values[I] = Accelerometer_values[I]/16384.0; // The scale is 2G and vary
        between -32,764 - 32,764 meaning 1G in raw data = 16384
195 }
196
197 // Calc angle from MPU 0
198 float accAngle = (atan2(Accelerometer_values[1], Accelerometer_values[0]) * 180 / PI) + 180;
        // x and y values from the accelerometer. Then converts from radians to degrees.
199     // +180 moves the seam to the bottom of the wheel rather than having to handle
        negative logic in later code.
200 if (accAngle > 180) accAngle -= 360; //Move the seam to the bottom of the steeringwheel so
        that math is easier
201
202
203 // Get Gyro Rate (The "Fast" Motion)
204 // Scale factor for MPU6500 at 500deg/s is 65.5
205 float gyroRate = (mpu.gz - gyroBias) / 65.5;
206
207 // Filter to keep steering smooth and remove accelerometer noise
208 // New Angle = 98% of (Old Angle + Gyro change) + 2% of (Accelerometer Angle)
209 filteredAngle = 0.98 * (filteredAngle + gyroRate * dt) + 0.02 * (accAngle);
210
211
212 //----- Button Press Logic -----
213 // Button Press Logic for reset button
214 if (digitalRead(PIN_RESET_BTN) == LOW){
215     zeroAngle = filteredAngle; // zeroed angle is set to current angle
216     delay(20); //debounce delay
217 }
218
219 // Button Press Logic for reset button
220 if (digitalRead(PIN_CALIBRATE_BTN) == LOW){
221     calibrateGyro(); // gyro bias is reset in order to recalibrate on flat surface
222     delay(20); //debounce delay
223 }
224
225 //calculate the angle from the zeroed angle
226 float correctAngle = filteredAngle - zeroAngle;
227 int steerVal = (int)constrain(correctAngle, -90, 90); // Hard limit for steering angle (-90,
        90) degree range
228
229
230 // Format structured data
231 strcpy(myData.a, "STEER"); // Message for receiver to know which data it is receiving
232 myData.b = steerVal; // Steering angle for the rover to interpret (-90,90) range
233 myData.c = 0; // Sending no c Data
234
235
236 // Send message via ESP-NOW every 50 ms non interfering (To prevent issues with the
        gyroscope filter integration)
237 if (millis() - lastDelayPrintTime > 50) { //only send every 50ms
238     esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));
239     if (result != ESP_OK) {
240         Serial.println("Sending_error");
241     }
242     lastDelayPrintTime = millis(); //reset last time of data sent
243 }
244

```

```

245 //----- Print For Testing (Non-interfering)
246 if (millis() - lastPrintTime > 100) { // Only print every 100ms
247     //print 3 steering values for debugging
248     Serial.print("Acc:"); Serial.print(accAngle);
249     Serial.print("_Filtered:"); Serial.print(correctAngle);
250     Serial.print("_Steer:"); Serial.println(steerVal);
251     lastPrintTime = millis(); //reset last time of print
252 }
253 }
254
255
256
257
258 //----- END OF SKETCH -----
259 //----- END OF SKETCH -----
260 //----- END OF SKETCH -----

```

Code 10: Rover Receiver and Serial Bridge

```
1  /*
2  * -----
3  * PROJECT: Project C.A.R.S. | Wireless Rover Control
4  * MODULE:  Rover Receiver & Serial Bridge
5  * AUTHOR:  Lawton Jones
6  * COLLEGE: University of Vermont | Dept. of Mechanical Engineering
7  * COURSE:  CMPE 3815: Microcontrollers
8  * DATE:    4 May 2026
9  * -----
10 * * DESCRIPTION:
11 * This script acts as the central communication hub for the Rover. It
12 * receives wireless control packets from multiple transmitters via ESP-NOW,
13 * aggregates the inputs (Gas, Brake, and Steering), and relays them to
14 * the primary motor control unit (Arduino) via a packetized UART stream.
15 * * OPERATION LOGIC:
16 * 1. Wireless Gateway: Listens for ESP-NOW traffic and utilizes a callback
17 * function to differentiate between "PEDALS" and "STEER" data sources.
18 * 2. Data Synchronization: Stores the most recent 8-bit pedal values and
19 * steering angles into global variables for asynchronous processing.
20 * 3. Serial Framing: Encapsulates the control data into a delimited packet
21 * format: "<rPedal,lPedal,steerAngle>" to ensure data integrity.
22 * 4. Hardware Bridge: Transmits the formatted string over HardwareSerial1
23 * (UART1) at 9600 Baud to the downstream microcontroller every 50ms.
24 * * HARDWARE CONFIGURATION:
25 * - MCU: ESP32 (Receiver)
26 * - Communication: UART1 (HardwareSerial) @ 9600 Baud
27 * - Power: 7.4V Battery Pack (common ground with Arduino and ESP32)
28 * - Pin 17: RX (Receive from Arduino)
29 * - Pin 16: TX (Transmit to Arduino)
30 * - Protocol: ESP-NOW (Wi-Fi Station Mode)
31 * -----
32 * SOURCES:
33 * - https://copperhilltech.com/blog/esp32-serial-ports-uart0-uart1-uart2-access-using-the-arduino-ide/
34 */
35
36
37
38
39 //----- LIBRARIES -----
40 #include <Arduino.h>
41 #include <HardwareSerial.h>
42 #include <esp_now.h>
43 #include <WiFi.h>
44
45
46
47
48 //----- INIT -----
49 //Assign UART1 and init UART1 info (ESP has multiple Hardware Serials)
50 HardwareSerial mySerial(1);
51
52
53 //define serial data and pins
54 #define RxPIN      17
55 #define TxPIN      16
56 #define BAUDRATE    9600
57 #define SER_BUF_SIZE 1024
58
```

```

59
60
61 //init values that rover will receive for driving control
62 int steerVal = 0;
63 int gasVal = 0;
64 int brakeVal = 0;
65
66
67 //----- Message Structure -----
68 // Define a data structure **Must be consistent between all communicating ESPs
69 // a = ESP SENDING CODE
70 // b = GAS PEDAL VALUE / STEERING VALUE
71 // c = BRAKE PEDAL VALUE
72 typedef struct struct_message {
73     char a[32];
74     int b;
75     int c;
76 } struct_message;
77
78 // Create a structured object
79 struct_message myData;
80
81
82 // Callback function executed when data is received
83 void OnDataRecv(const esp_now_recv_info_t * rcv_info, const uint8_t *incomingData, int len) {
84     memcpy(&myData, incomingData, sizeof(myData));
85
86     //based on data recieved we set the gas, brake and steering values to be sent to the arduino
87
88     //DATA FROM PEDALS
89     if(strcmp(myData.a, "PEDALS") == 0){        // Check for message from pedal transmitter
90         gasVal = myData.b;           // GasPedal Value from the pedals (0,255) range
91         brakeVal = myData.c;         // Brake Pedal value from pedals (0,255) range
92         // //serial print to monitor values received for debugging
93         // Serial.print("Data Received-- ");
94         // Serial.print("Gas: ");
95         // Serial.print(gasVal);
96         // Serial.print(" | Brake: ");
97         // Serial.println(brakeVal);
98     }
99     //DATA FROM STEERING
100    else if(strcmp(myData.a, "STEER") == 0){        //check for message from steering transmitter
101        steerVal = myData.b;           //Steering Value from Steering unit (-90, 90) range
102        // //serial print steering value to monitor for debugging
103        // Serial.print("Steer: ");
104        // Serial.println(steerVal);
105    }
106 }
107 }
108
109
110
111
112
113
114 //----- SETUP -----
115 void setup() {
116     //Setup Serial monitor and Serial Communication
117     Serial.begin(115200);    //begin normal Serial Monitor at baud rate similar to all esp32s
118
119     mySerial.setRxBufferSize(SER_BUF_SIZE);    //standard arduino has 64 bytes
120                                                // ESP32 has 256 bytes
121                                                //Call must come before begin()
122     //Initialize the Serial Port (UART1) -- for communication with Arduino

```

```

123 mySerial.begin(BAUDRATE, SERIAL_8N1, RxPIN, TxPIN);
124
125
126 // Set ESP32 as a Wi-Fi Station
127 WiFi.mode(WIFI_STA);
128
129 // Initilize ESP-NOW
130 if (esp_now_init() != ESP_OK) {
131     Serial.println("Error_initializing_ESP-NOW");
132     return;
133 }
134
135 // Register callback function
136 esp_now_register_recv_cb(OnDataRecv);
137
138
139 }
140
141
142
143
144
145 //----- LOOP -----
146 void loop() {
147     //load variables to send to arduino
148     int rPedal = gasVal;           // value 0-255, 255 is fully pressed
149     int lPedal = brakeVal;         // value 0-255, 255 is fully pressed
150     int steerAngle = steerVal;     // -90 to 90 for steering wheel. 90 deg ccw turn is -90, 90
                                   // deg cw turn is 90
151
152     //Send data through serial hardware in Packets for the arduino to interpret <rPedal,lPedal,
                                   //steerAngle>
153     mySerial.print("<");
154     mySerial.print(rPedal);
155     mySerial.print(",");
156     mySerial.print(lPedal);
157     mySerial.print(",");
158     mySerial.print(steerAngle);
159     mySerial.print(">");
160
161     //Serial.print("Data Sent");
162     delay(50); //50ms refresh rate for the data to send
163 }
164
165
166
167 //----- END OF SKETCH -----
168 //----- END OF SKETCH -----
169 //----- END OF SKETCH -----

```

Code 11: Rover Motor Actuation and Driving Logic

```
1  /*
2  * -----
3  * PROJECT: Project C.A.R.S. | Wireless Rover Control
4  * MODULE:  Rover Motor Actuation & Driving Logic
5  * AUTHOR:  Lawton Jones
6  * COLLEGE: University of Vermont | Dept. of Mechanical Engineering
7  * COURSE:  CMPE 3815: Microcontrollers
8  * DATE:    4 May 2026
9  * -----
10 * * DESCRIPTION:
11 * This script serves as the primary power and movement controller for the
12 * rover. It decodes the telemetry packets received from the ESP32 serial
13 * bridge and translates those control signals into physical movement via
14 * an L293D motor driver shield and a four-motor drivetrain.
15 * * OPERATION LOGIC:
16 * 1. Packet Parsing: Listens on SoftwareSerial for the '<' start delimiter
17 * and parses the CSV-formatted gas, brake, and steering telemetry.
18 * 2. Velocity Calculation: Derives a net 'speedVal' by subtracting brake
19 * input from gas input to determine forward/reverse magnitude.
20 * 3. Kinematic Mapping: Implements a three-tier steering logic:
21 * - Deadzone ( 10 ): Maintains synchronized RPM for straight travel.
22 * - Differential Turn (10 -50 ): Scales inner wheel speed relative to
23 * the steering angle for smooth arc turns.
24 * - Pivot/Rotate (>50 ): Executes zero-radius tank turns by reversing
25 * the rotation of inner wheels.
26 * 4. Actuation: Interfaces with an 8-bit shift register to set H-bridge
27 * directions and applies PWM signals to regulate motor torque.
28 * * HARDWARE CONFIGURATION:
29 * - MCU: Arduino (Motor Controller)
30 * - Motor Driver: L293D Driver Shield (Shift Register Interface)
31 * - Comms: SoftwareSerial @ 9600 Baud (RX: A0 | TX: A1)
32 * - PWM Pins: 3, 5, 6, 11 (Motor Speeds)
33 * - Shift Register: Pins 4 (CLK), 7 (EN), 8 (DATA), 12 (LATCH)
34 * -----
35 */
36
37
38
39
40
41 //----- LIBRARIES -----
42 #include <Arduino.h>
43 #include <SoftwareSerial.h>
44
45
46
47
48 //----- INIT -----
49 //init Serial at analog pins
50 #define RxPIN    A0
51 #define TxPIN    A1
52
53 //Set up Software Serial Monitor for ESP communication with new pins
54 SoftwareSerial mySerial(RxPIN, TxPIN); // RX, TX
55
56 // init variables that will be received for driving
57 int gasVal;
58 int brakeVal;
59 int steerAngle;
```

```

60
61
62
63 //----- MOTOR INIT -----
64 //Pin defintions
65 const int PWM2A = 11;      //M1 motor PWM signal pin
66 const int PWM2B = 3;      //M2 motor PWM signal pin
67 const int PWM0A = 6;      //M3 motor PWM signal pin
68 const int PWM0B = 5;      //M4 motor PWM signal pin
69 const int DIR_CLK = 4;     // Data input clock line
70 const int DIR_EN = 7;     //Equip the L293D enabling pins
71 const int DATA = 8;      // USB cable
72 const int DIR_LATCH = 12;  // Output memory latch clock
73
74
75 //Binary Values of the motor directions in order to turn the motors the directions needed to
    turn to go the direction
76 const int Move_Forward = 39;      //Move Forward
77 const int Move_Backward = 216;    //Move Backward
78 const int Left_Move = 116;        //Left translation
79 const int Right_Move = 139;       //Right translation
80 const int Right_Rotate = 149;     //Rotate Right
81 const int Left_Rotate = 106;      //Rotate Left
82 const int Stop = 0;               //Stop
83 const int Upper_Left_Move = 36;    //Upper Left Move
84 const int Upper_Right_Move = 3;    //Upper Right Move
85 const int Lower_Left_Move = 80;    //Lower Left Move
86 const int Lower_Right_Move = 136;  //Lower Right Move
87 const int Drift_Left = 20;         //Drift on Left
88 const int Drift_Right = 10;        //Drift on Right
89
90 //Speed of motors using pwm
91 int Speed1 = 255; //Set the default speed between 1 and 255
92 int Speed2 = 255; //Set the default speed between 1 and 255
93 int Speed3 = 255; //Set the default speed between 1 and 255
94 int Speed4 = 255; //Set the default speed between 1 and 255
95 //-----
96
97
98
99
100 //----- MOTOR FUNCTION -----
101 //(input direction(binary val for shift reg), Motor Speeds 1,2,3,4)
102 void Motor(int Dir, int Speed1, int Speed2, int Speed3, int Speed4) {
103     analogWrite(PWM2A, Speed1); //Motor PWM speed regulation
104     analogWrite(PWM2B, Speed2); //Motor PWM speed regulation
105     analogWrite(PWM0A, Speed3); //Motor PWM speed regulation
106     analogWrite(PWM0B, Speed4); //Motor PWM speed regulation
107
108     digitalWrite(DIR_LATCH, LOW); //DIR_LATCH sets the low level and writes the
        direction of motion in preparation
109     shiftOut(DATA, DIR_CLK, MSBFIRST, Dir); //Write Dir motion direction value
110     digitalWrite(DIR_LATCH, HIGH); //DIR_LATCH sets the high level and outputs the
        direction of motion
111 }
112
113
114
115
116
117 //----- SETUP -----
118 void setup() {
119     Serial.begin(9600); // begin serial monitor for computer communication

```

```

120 mySerial.begin(9600); // begin the serial communication between the ESP and Arduino (
    Software Serial)
121
122
123 //Setup Pin Modes for Motor Shield all OUTPUT
124 pinMode(DIR_CLK, OUTPUT);
125 pinMode(DATA, OUTPUT);
126 pinMode(DIR_EN, OUTPUT);
127 pinMode(DIR_LATCH, OUTPUT);
128
129 pinMode(PWM0B, OUTPUT);
130 pinMode(PWM0A, OUTPUT);
131 pinMode(PWM2A, OUTPUT);
132 pinMode(PWM2B, OUTPUT);
133 }
134
135
136
137
138
139 //----- LOOP -----
140 void loop() {
141
142 //----- pull data from esp -----
143 //if there is data in the serial monitor
144 if (mySerial.available() > 0) {
145     char startChar = mySerial.read(); //read serial data from esp
146
147     // Check if this is the start of our data packet
148     // if the data is formatted in the correct form read the driving control variables from
        esp
149     if (startChar == '<') {
150         gasVal = mySerial.parseInt(); // Reads up to the first comma,
151         brakeVal = mySerial.parseInt(); // Reads up to the second comma
152         steerAngle = mySerial.parseInt(); // Reads up to the '>'
153
154         // Print the received data on the computer serial monitor for debugging
155         //Serial.print("Gas: "); Serial.print(gasVal);
156         //Serial.print(" | Brake: "); Serial.print(brakeVal);
157         //Serial.print(" | Steer: "); Serial.println(steerAngle);
158     }
159 }
160
161
162
163 //----- DRIVING LOGIC -----
164 //----- take data and actuate motor as needed -----
165
166 // Steup steering zone consts
167 const int DEADZONE_THR = 10;
168 const int SHARP_TURN_THR = 50;
169
170 // take difference of gas and brake pedal to get speed value: FWD(+), BWD(-)
171 int speedVal = gasVal - brakeVal;
172 //---- STOPPED----
173 if (speedVal == 0){
174     Motor(Stop, 0, 0, 0, 0);
175 }
176
177 //---- FORWARD ----
178 else if (speedVal > 0){
179     //---- STRAIGHT FWD ----
180     // if steer anlge in the straight zone

```

```

181     if(steerAngle <= DEADZONE_THR && steerAngle >= -DEADZONE_THR){           // Create deadzone
182         range of angle of steering wheel. 10 deg in each direction
183         Motor(Move_Forward, speedVal, speedVal, speedVal, speedVal);
184     }
185     //---- FWD RIGHT ----
186     // if the steer angle in the slow right turn zone
187     else if (steerAngle > DEADZONE_THR && steerAngle <= SHARP_TURN_THR){
188         int outerSpeed = speedVal;
189         int innerSpeed = map(steerAngle, DEADZONE_THR, SHARP_TURN_THR, 255, 0); //Convert the
            steering angle to the inner wheel speed: 10deg small turn high speed, 60 deg sharp
            turn low speed
189         Motor(Move_Forward, outerSpeed, innerSpeed, innerSpeed, outerSpeed); //Motors 2,3 are
            the right motors(inner) and 1,4 are the left motors(outer)
190     }
191     //---- ROTATE RIGHT ----
192     // if the steer angle in the sharp right turn zone
193     else if (steerAngle > SHARP_TURN_THR){
194         int outerSpeed = speedVal;
195         int innerSpeed = map(steerAngle, SHARP_TURN_THR, 90, 0, 100); //Convert the steering
            angle to the inner wheel speed: 60deg small turn no inner speed, 90deg sharp turn
            high BWD inner speed
196         //(Not Max it feels to aggressive 100 is sweet spot of not too powerful but still
            sharp enough)
197         Motor(Right_Rotate, outerSpeed, innerSpeed, innerSpeed, outerSpeed); //Motors 2,3 are
            the right motors(inner) and 1,4 are the left motors(outer)
198     }
199     // ---- FWD LEFT ----
200     //if the steer angle in the slow left turn zone
201     else if (steerAngle < -DEADZONE_THR && steerAngle >= -SHARP_TURN_THR){
202         int outerSpeed = speedVal;
203         int innerSpeed = map(steerAngle, -DEADZONE_THR, -SHARP_TURN_THR, 255, 0); //Convert the
            steering angle to the inner wheel speed: 10deg small turn high speed, 90deg sharp
            turn low speed
204         Motor(Move_Forward, innerSpeed, outerSpeed, outerSpeed, innerSpeed); //Motors 2,3 are
            the right motors(outer) and 1,4 are the left motors(inner)
205     }
206     //---- ROTATE LEFT ----
207     // if the steer angle in the sharp left turn zone
208     else if (steerAngle < -SHARP_TURN_THR){
209         int outerSpeed = speedVal;
210         int innerSpeed = map(steerAngle, -SHARP_TURN_THR, -90, 0, 100); //Convert the steering
            angle to the inner wheel speed: 60deg small turn no inner speed, 90deg sharp turn
            high BWD inner speed
211         //(Not Max it feels to aggressive 100 is sweet spot of not too powerful but still
            sharp enough)
212         Motor(Left_Rotate, innerSpeed, outerSpeed, outerSpeed, innerSpeed); //Motors 2,3 are
            the right motors(outer) and 1,4 are the left motors(inner)
213     }
214 }
215
216
217 //---- BACKWARD ----
218 // if the speed value is negative
219 else if(speedVal < 0){
220     int reverseVal = abs(speedVal); // convert the negative speed val to positive to be
        handed to the motors correctly
221     //---- STRAIGHT BWD----
222     // if the steer angle in the straight zone
223     if(steerAngle <= DEADZONE_THR && steerAngle >= -DEADZONE_THR){           // Create deadzone
        range of angle of steering wheel. 10 deg in each direction
224         Motor(Move_Backward, reverseVal, reverseVal, reverseVal, reverseVal);
225     }
226     //(Backwards turning only has slower turn modes to make driving easier)
227     //---- BWD RIGHT ----

```

```

228 // if the steer angle in the right turn zone
229 else if (steerAngle > DEADZONE_THR){
230     int outerSpeed = reverseVal;
231     int innerSpeed = map(steerAngle, DEADZONE_THR, 90, 255, 0); //Convert the steering
        angle to the inner wheel speed: 10deg small turn high speed, 90 deg sharp turn low
        speed
232     Motor(Move_Backward, outerSpeed, innerSpeed, innerSpeed, outerSpeed); //Motors 2,3 are
        the right motors(inner) and 1,4 are the left motors(outer)
233 }
234 //---- BWD LEFT ----
235 // if the steer angle in the left turn zone.
236 else if (steerAngle < -DEADZONE_THR){
237     int outerSpeed = reverseVal;
238     int innerSpeed = map(steerAngle, -DEADZONE_THR, -90, 255, 0); //Convert the steering
        angle to the inner wheel speed: 10deg small turn high speed, 90 deg sharp turn low
        speed
239     Motor(Move_Backward, innerSpeed, outerSpeed, outerSpeed, innerSpeed); //Motors 2,3 are
        the right motors(outer) and 1,4 are the left motors(inner)
240 }
241 }
242
243
244 }
245
246
247
248 //----- END OF SKETCH -----
249 //----- END OF SKETCH -----
250 //----- END OF SKETCH -----

```

INITIAL PROJECT PROPOSAL

CONCEPT

For this lab activity, we will design a wireless control system for the Lafvin Mecanum car through the use of ESP32 microcontrollers using their wireless communication protocols. We will use a custom steering wheel and Logitech foot pedals as inputs to control the rover. This project will have a focus on wireless communication and speaking with unknown input devices through the use of ESP32s.

REQUIREMENTS

- **Wireless Communication:** Implement wireless communication from control input to rover output through the ESP32.
- **Speed Mapping:** Map input from Pedals to rover speed control. Speed is variable: small pedal angle → slow speed, large pedal angle → fast speed.
- **Proportional Steering Mapping:** Proportionally map steering wheel angle input to rover turn radius. Steering feels smooth and not choppy.
- **Fail-Safe:** Implement fail safe to stop rover if wireless connection is lost.
- **Sensor Drift Accuracy and Steering Calibration:** Implement steering zeroing reset button and correctly handle drift from MPU by maintaining $\pm 2^\circ$ over 5 min of driving.

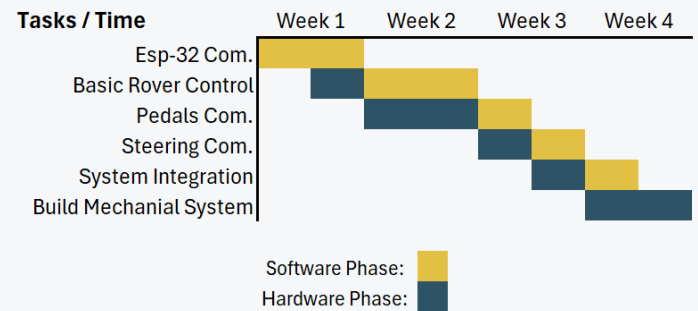
HARDWARE

- ESP32 DevKit (3)
- Mecanum Wheel Car Kit
- MPU6500 Sensor for Steering
- Logitech Pedals with DB9 Connector
- Wheel and Pedal Wire Housing

STRETCH GOALS

- **Dashboard Display:** Implement OLED Dashboard to display speed and turning angle of car, with other information available.
- **Driving Modes:** Implement different driving modes through steering interface: Reverse, Crab-walk, etc.
- **Active Electronic Stability Control (ESC):** Implement an autonomous controller that accounts for the variation in friction and force from wheels to move car in straight line if deviating from straight.

TIMELINE



MILESTONES

Week 1: Foundation

- Wireless communication between ESP32s
- Wire ESP32 to Lafvin rover
- Establish basic rover control

Week 2: Inputs

- Communicate with Logitech pedals
- Communicate with steering wheel

Week 3: Integration

- Implement control scheme
- Calibrate system

Week 4: Refinement

- Finalize mechanical system
- Add extra functionality

QUESTIONS

- How will the pedals be wired? What type of communication protocol do they use?
- How do I control the steering of the rover at different speed inputs compared to full motor speed?
- How do I account for sensor drift within MPU and its effect on steering?